IHUD-BERT: A Large-Scale Network Traffic Classification Method Based on Pre-Training Transformers and Knowledge Distillation

Yahui Hu, Tiancun Deng, Junping Song, Pengfei Fan, and Xu Zhou

***Abstract*—Accurate and efficient classification of large-scale network traffic is fundamental to ensuring network Quality of Service (QoS) and security. However, existing methodologies face critical challenges. While payload-based pre-trained models have achieved success, they often rely on memorizing session-specific encryption artifacts, leading to catastrophic performance degradation under realistic "inter-flow" splitting scenarios. Furthermore, the scarcity of labeled samples and the high computational cost of large models hinder their practical deployment. To address these limitations, this paper proposes an integrated framework, IHUD-BERT, which integrates robust feature representation with knowledge distillation. We first introduce the IP Header Unit (IHU) scheme, applying a sliding window over IP packet headers. Unlike payload-based approaches, this method captures invariant behavioral patterns that remain consistent across encrypted sessions, ensuring robustness against encryption randomization while augmenting limited training data. Subsequently, a pre-trained Transformer (IHU-BERT) is constructed as a teacher model, and its comprehensive knowledge is transferred to a lightweight student model (IHUD-BERT) via hierarchical knowledge distillation. Comprehensive evaluations on multiple datasets validate our approach. The results demonstrate that while payload-based baselines collapse under strict inter-flow evaluation on encrypted traffic, our method maintains high accuracy. Notably, the lightweight IHUD-BERT achieves only a marginal performance degradation compared to its teacher counterpart while delivering millisecond-level inference latency. These results establish the proposed framework as a robust, high-precision, and efficient solution for modern encrypted traffic classification.**

***Index Terms*—Traffic classification, pre-training, knowledge distillation, IP header, sliding window.**

Received 24 August 2025; revised 19 December 2025, 22 March 2026, and 9 May 2026; accepted 15 May 2026. Date of publication 22 May 2026; date of current version 29 May 2026. This work was supported by the the 2026 Enhancement of Independent Innovation - Disciplinary Interdisciplinary Innovation Project through the project Construction of Multi-mode Fusion Communication Platform for Mine Leaky Cable (81001203A56), the Fundamental Research Funds for the Central Universities (2025ZKPYZN02), the National Key R&D Program of China (2024YFB2908700), the Youth Innovation Promotion Association of Chinese Academy of Sciences (2021168), and Guoneng Zhishen Control Technology Co., Ltd. through the project Research and Development of Boundary-aware Intelligent Coal Sorting Technology Using Weakly Supervised Training. The associate editor coordinating the review of this article and approving it for publication was E. Oki. *(Corresponding author: Tiancun Deng.)*

Yahui Hu and Tiancun Deng are with China University of Mining and Technology-Beijing, Beijing 100083, China (e-mail: huyahui@cumtb.edu.cn; dengtiancun.cumtb@gmail.com).

Junping Song, Pengfei Fan, and Xu Zhou are with the Computer Network Information Center, Chinese Academy of Sciences, Beijing 100190, China (e-mail: songjunping@cnic.cn; fanpengfei@cnic.cn; zhouxu@cnic.cn).

Digital Object Identifier 10.1109/TCCN.2026.3695843

I. Introduction

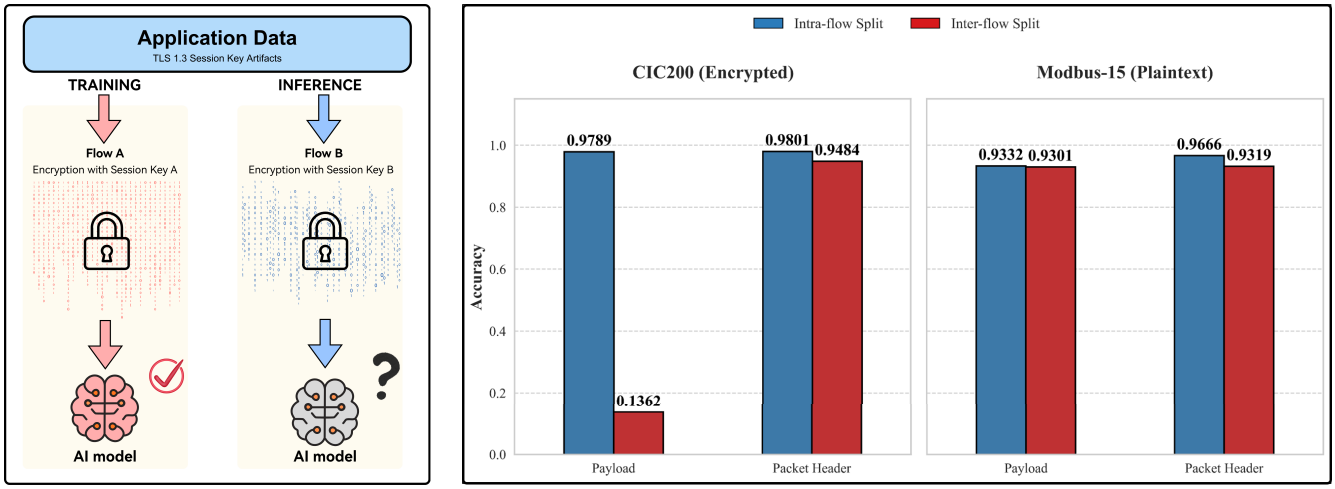
TRADITIONAL network traffic classification methods, including port-based and Deep Packet Inspection (DPI), have been challenged by dynamic port allocation and the proliferation of end-to-end encryption [1]. While conventional Machine Learning(ML) approaches to bypass payload dependency, they still struggle with feature engineering costs and the evolving nature of encrypted protocol [2].

To overcome the constraints of feature engineering, research focus shifted to Deep Learning (DL) techniques such as Convolutional and Recurrent Neural Networks (CNNs and RNNs), which can automatically learn hierarchical feature representations from raw traffic data [3]. Building on this foundation, and inspired by successes in Natural Language Processing (NLP) [4], pre-trained models based on the Transformer architecture have been introduced to the field [5], [6]. These models learn generic "behavioral paradigms" from vast, unlabeled traffic datasets and can be fine-tuned to achieve state-of-the-art performance on specific downstream tasks. Despite the remarkable capabilities of existing pre-trained models, their application to network traffic classification faces three critical challenges that hinder practical deployment.

First, the validity of payload-based feature extraction is fundamentally compromised by modern encryption protocols. While existing studies often report high accuracy [5], our empirical analysis reveals that these results are largely inflated by "session artifacts" rather than stemming from generalized feature learning. This overestimation is critically dependent on the sample processing strategy employed. Conventional "intra-flow" (packet-level) splitting randomly partitions packets from the same session across both training and testing sets. This overlap allows models to memorize session-specific encryption patterns (e.g., ephemeral keys) found in both sets. In sharp contrast, strict "inter-flow" (session-level) splitting enforces a hard boundary where complete sessions are assigned exclusively to either the training or testing set. This ensures that the testing data consists of entirely unseen flows with different encryption parameters, faithfully simulating real-world deployment. As illustrated in Fig. 1a, while payload-based models thrive under the permissive intra-flow setting, their performance collapses significantly under the realistic inter-flow protocol, exposing the inherent inapplicability of payload features for encrypted traffic.

When evaluating the payload-based ET-BERT [5] on encrypted data, its accuracy precipitates from

2332-7731 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.



(a) Limitations of Intra-flow Classification **(b)** Empirical Validation

Fig. 1. The impact of encryption artifacts on model generalization. (a) Limitations of intra-flow classification: A conceptual illustration showing how payload-based models memorize ephemeral session keys (visualized as distinct textures) during intra-flow training. This results in "data leakage," where the model fails to generalize to unseen flows (Inter-flow inference) due to domain shifts caused by new encryption keys. (b) Empirical validation: Experimental results on the encrypted CIC200 dataset confirm a performance collapse for ET-BERT under strict inter-flow splitting. However, the high accuracy on the plaintext Modbus-15* dataset proves that this failure is induced specifically by encryption. Crucially, our header-based approach (IHU) remains robust in all settings, demonstrating that header behaviors are consistent and encryption-agnostic.

97.89%(intra-flow) to approximately 13.76%(inter-flow) as empirically verified in Figure 1b. In contrast, our header-based approach maintains superior robustness across all scenarios, proving its resilience to encryption randomization. To confirm that this degradation is attributable to encryption rather than the inherent complexity of the inter-flow task, we conducted a control experiment using the Modbus-15.[1] As shown in Figure 1b, payload-based models generalized well on plaintext Modbus traffic even under inter-flow splitting. This isolates encryption as the root cause of the performance degradation in CIC200. This is because IP header is plaintext and header-based models can learn traffic behavior patterns from the header (e.g., specific service requests often send initial packets of a fixed size, or follow a particular sequence of message exchanges). Since these behavioral patterns are characteristics of the application itself, and not products of the session key, they maintain consistency and robustness across different sessions (i.e., under strict Inter-flow splitting).

Second, there is a data scarcity problem. The immense cost and difficulty of collecting and labeling large-scale, real-world network data across hundreds of application classes mean most public datasets are limited in either variety or sample volume [8]. This data deficiency, combined with a lack of effective data augmentation techniques, results in models with low accuracy and poor usability, presenting a major hurdle for both academia and industry [9]. Third, pre-trained models suffer from excessive computational complexity [5]. Their powerful representational capabilities are supported by enormous parameter counts and high computational costs, which prohibit their deployment on resource-constrained network hardware such as routers, switches, and Internet of Things(IoT) gateways. This creates a significant gap between high-performance academic models and the industrial demand for low-cost, high-efficiency solutions.

To address these challenges, we propose a systematic framework that synergizes robust feature representation with model compression. Capitalizing on the insight that packet header behaviors remain unencrypted and highly consistent across sessions even when payloads are randomized, we introduce the IP Header Unit (IHU) as a robust and session-invariant feature input, which effectively captures consistent behavioral patterns across different encrypted sessions. Furthermore, to bridge the gap between high-performance pre-training and practical deployment on resource-constrained devices, we employ a knowledge distillation strategy to derive a lightweight student model, IHUD-BERT(IHU Distilled-BERT), from a large-scale pre-trained teacher IHU-BERT.

The main contributions of this paper are as follows:

First, we propose the IHU scheme, which employs a strategic sliding window mechanism exclusively over IP packet headers. This mechanism serves as an effective data augmentation technique to mitigate the scarcity of labeled data. Unlike payload-based methods that are vulnerable to session-specific encryption artifacts, our approach captures invariant behavioral features. Consequently, it maintains high accuracy even under strict inter-flow splitting scenarios where traditional payload-based baselines experience significant performance collapse.

Second, we design a two-stage "pre-training and knowledge distillation" framework to achieve lightweight deployment. By transferring comprehensive knowledge from the high-performance teacher (IHU-BERT) to the streamlined student (IHUD-BERT), we effectively solve the deployment bottleneck. This distillation-based approach ensures that the student model retains the teacher's representational power while meeting the stringent efficiency requirements of edge network devices.

[1]To isolate the impact of encryption, we employed the unencrypted CIC Modbus Dataset 2023 [7] as a control group. We constructed the Modbus-15 dataset by extracting the first 10,000 packets from each flow across 15 categories. Its consistent plaintext payload allows us to confirm that the performance degradation observed in other datasets is specifically induced by encryption artifacts.

Finally, comprehensive evaluations on multiple large-scale datasets validate the superiority of our framework. The results demonstrate that our teacher model surpasses existing state-of-the-art methods, while the lightweight IHUD-BERT achieves an optimal trade-off, maintaining near-teacher accuracy while significantly accelerating inference speed.

The remainder of this paper is organized as follows. Section II reviews related work on network traffic classification, pre-trained models applied in this domain, and knowledge distillation techniques. Section III elaborates on our proposed model design, including the IHU representation method, the core IHU-BERT pre-training architecture, and the IHUD-BERT knowledge distillation framework for generating the lightweight model. Section IV presents the experimental evaluation, first introducing the datasets, evaluation metrics, and experimental setup, followed by a presentation and analysis of the performance of IHU-BERT and IHUD-BERT. Finally, Section V discusses and summarizes our work and looks forward to future research directions.

II. Related Works

This section reviews three key areas closely related to our research: the evolution of network traffic classification techniques, the use of pre-trained models for enhancing traffic representation, and the application of knowledge distillation for developing efficient and deployable models.

A. Network Traffic Classification

1) The Evolution of Classification Techniques: The development of network traffic classification technology has progressed through four primary stages: port-based methods, DPI-based methods, traditional ML-based methods, and DL-based methods.

Early attempts at traffic classification primarily relied on well-known port numbers assigned by the Internet Assigned Numbers Authority (IANA), such as port 80 for HTTP and port 25 for SMTP [10]. While offering simplicity and low overhead, these methods have been rendered unreliable in modern environments by dynamic port allocation and obfuscation techniques. Subsequently, DPI emerged as a more powerful technique. DPI identifies signatures or patterns associated with specific applications or protocols by analyzing the payload content of data packets. By examining the actual data content and matching 'fingerprints' (specific strings or bitstreams) against a signature library, DPI provided high classification accuracy at the time [11]. However, the widespread adoption of encryption protocols such as TLS/SSL has greatly diminished the effectiveness of DPI, as encryption renders packet payloads opaque. As methods dependent on payload content waned, machine learning techniques began to gain prominence, with the core idea of classifying traffic based on statistical properties that are generally independent of encryption. These features include packet sizes, inter-arrival times, flow duration, byte counts, and other time or volume-related statistics [2]. Despite this, traditional ML-based methods are highly dependent on expert-driven feature engineering. This process is not only time-consuming but also domain-specific, making feature engineering itself a new bottleneck [1].

The emergence of deep learning models, particularly the application of CNNs and RNNs, brought about a paradigm shift in traffic classification. DL models can automatically learn hierarchical features from raw or minimally processed traffic data, thereby reducing the reliance on manual feature engineering [3]. This has enabled DL models to achieve state-of-the-art performance in various traffic classification tasks. However, deep learning methods also face their own set of challenges. First, DL models typically require large labeled datasets for training, which are often scarce or expensive to acquire in the networking domain. Second, they are prone to learning biased representations from imbalanced data and may struggle to adapt to new or unseen traffic patterns without retraining [12]. Furthermore, many early DL methods still relied on payload data, thereby inheriting some of the limitations of DPI when handling encrypted traffic. The IHU method proposed in this study aligns with this trend by using only the first 12 bytes of the IP header for analysis. This approach eliminates the reliance on encrypted payloads, thereby avoiding the risk of models overfitting to session-specific encryption artifacts. Instead, through a sliding window mechanism, it effectively captures robust, invariant behavioral features to acquire more complex representations from limited data.

2) Classification of Large-Scale Network Traffic Data: Much of the current academic research focuses on datasets with a limited number of classes, which may not fully reflect the diversity and complexity of real-world network environments. This study aims to address the classification problem for "large-scale network traffic data". Taylor et al. utilized a Random Forest algorithm based on statistical features to classify 110 mobile applications, achieving high accuracy [13]. Rezaei et al. employed a hybrid model of 1D-CNN and LSTM to classify 80 application categories based on packet and payload sequences within a session [14]. Notably, the AppClassNet dataset, proposed by Wang et al.(Huawei), provides an important benchmark for research on large-scale, commercial-grade application identification [15]. This dataset contains up to 500 application categories and focuses on classification using packet size and direction information from session prefixes. The release of AppClassNet spurred subsequent research, such as the lightweight CNN model, LEXNet [16], proposed by Fauvel et al., based on this dataset. These advancements highlight the "scalability gap" between academic research and real-world requirements—many existing methods that perform well on small datasets have yet to have their effectiveness or applicability validated when faced with the vast diversity of applications in operational networks. Ultimately, large-scale classification demands not only high accuracy but also imposes stringent requirements on model efficiency (e.g., model size and inference speed) to facilitate practical deployment. This is also one of the core motivations for employing knowledge distillation in our IHUD-BERT model.

B. Pre-Training Models

The advent of pre-training techniques, particularly pre-trained models based on the Transformer architecture, has revolutionized fields such as Natural Language Processing (NLP) and Computer Vision (CV). This paradigm is increasingly being applied to network traffic analysis, aiming to learn robust and generalizable representations from vast amounts of

unlabeled data to achieve superior performance on downstream classification tasks, especially when labeled data is limited. Pre-trained language models, represented by BERT, marked a watershed moment [4]. Following BERT, a series of improved models have emerged. For example, RoBERTa [17] optimized pre-training strategies, such as using dynamic masking and larger datasets; ALBERT [18] significantly reduced the number of model parameters through techniques like parameter sharing and matrix factorization of word embeddings; while ELECTRA [19] introduced a new pre-training paradigm, Replaced Token Detection, which transforms the generative task into a discriminative one, thereby improving pre-training efficiency and effectiveness. These advancements collectively demonstrate the powerful capability of self-supervised learning to learn universal representations from large, unlabeled corpora.

Although network traffic data is not linguistic text, it similarly exhibits sequential patterns and contextual dependencies, such as the sequential relationship of packets within a flow or the associations between byte patterns within a packet. Pre-training techniques can capture this inherent structural information from large-scale unlabeled traffic data, generating representations that can achieve good performance during the fine-tuning stage with only a small amount of task-specific labeled data [17], [20].

The PERT model proposed by He et al. was one of the first attempts to apply Transformer-based pre-training techniques to encrypted traffic classification [6]. However, PERT lacks pre-training tasks and input representations specifically designed for traffic characteristics, which may limit its generalization ability. The ET-BERT model proposed by Lin et al. [5] represents a significant advancement, adapting the BERT architecture to learn robust contextual representations directly from encrypted packet payloads. However, despite its high performance in intra-flow settings, the model's reliance on encrypted payload data renders it susceptible to overfitting session-specific artifacts, which severely limits its generalization capability across unseen flows. Furthermore, its large model structure poses challenges for practical deployment. To address these issues, this paper proposes the IHUD-BERT framework, an effective integration that utilizes robust IP header features and achieves lightweighting through knowledge distillation.

C. Knowledge Distillation

Although large pre-trained models have achieved remarkable performance, their substantial model size and high computational demands often become obstacles to their deployment in resource-constrained environments or low-latency applications, such as real-time network traffic classification. Knowledge Distillation (KD) has emerged as an effective model compression technique. Its objective is to transfer the knowledge learned by a large, complex "teacher model" to a smaller, more efficient "student model", aiming to maintain the original performance while reducing complexity. Hinton et al. systematically articulated the core idea of knowledge distillation [21]. They proposed that the student model learns by mimicking the softened class probabilities (i.e., "soft targets") generated by the teacher model. These soft targets carry more information about inter-class similarities than traditional hard labels (one-hot encoding), which is known as "Dark Knowledge".

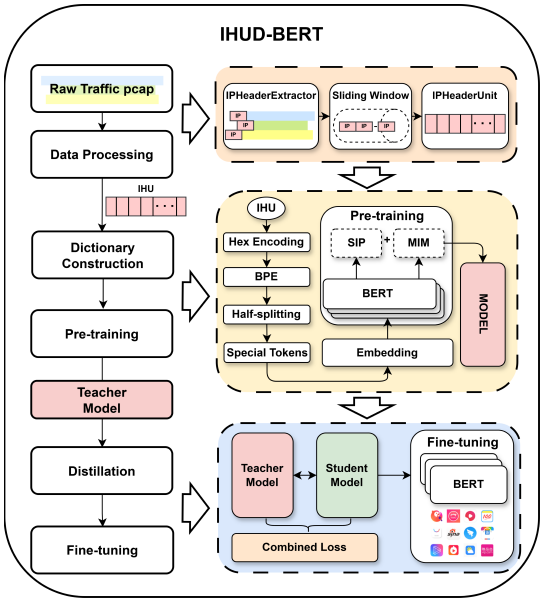


Fig. 2. Overview of the IHUD-BERT framework.

Knowledge distillation has been widely applied to compress large Transformer-based language models. TinyBERT [22] is a representative work in this area; it designed a comprehensive layer-by-layer distillation scheme for the Transformer architecture, transferring knowledge from the teacher model's embedding layer, attention matrices, hidden states, and final prediction layer to the student model. DistilBERT [23], on the other hand, employs a distillation strategy during the pre-training phase itself to directly train a general-purpose BERT model with fewer parameters. However, to the best of our knowledge, the combination of knowledge distillation and pre-training has not yet been applied to the field of large-scale network traffic classification. The IHUD-BERT model proposed in this study directly applies these principles. In our work, IHU-BERT (based on the BERT-base architecture) serves as the teacher model, while IHUD-BERT (based on the BERT-tiny architecture) acts as the student model.

III. Method

A. Workflow

This paper proposes a framework for large-scale network traffic classification based on a pre-trained Transformer and knowledge distillation, which we name IHUD-BERT. As illustrated in Figure 2, IHUD-BERT comprises five core modules: Data Processing, Dictionary Construction, Pre-training, Distillation, and Fine-tuning.

First, the framework extracts session flows from the raw network traffic. This paper introduces a specialized scheme that exclusively uses IP packet header bytes and employs a sliding window mechanism to sample over the packet sequence of each session flow, thereby enhancing the data for capturing temporal features. The data sampled from each window is aggregated into an IHU, which serves as the fundamental input unit for the model (Section III-B).

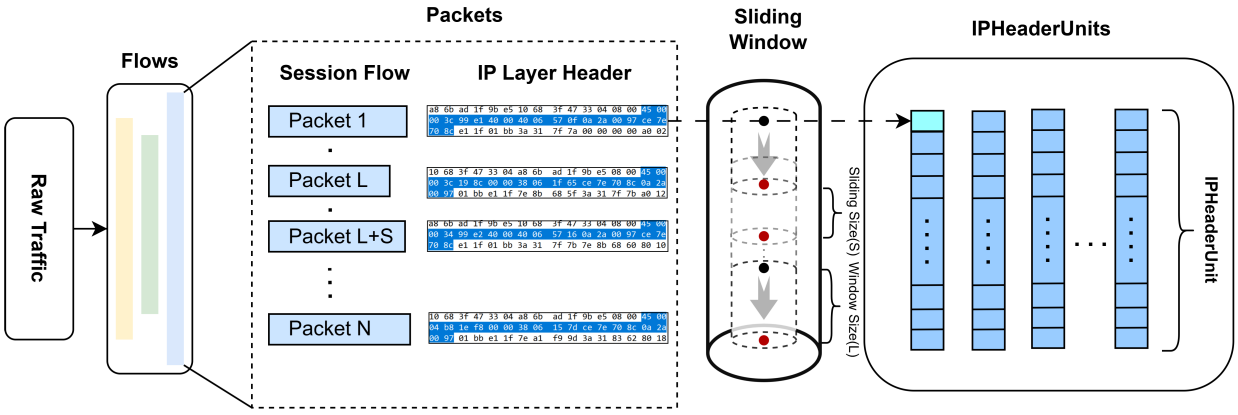


Fig. 3. Generation of an IHU via sliding window sampling.

The subsequently generated IHUs are first passed through the Dictionary Construction module to be converted into token sequences that the pre-trained model can process (Section III-C). These token sequences are then input into a large Transformer model based on the BERT architecture for self-supervised pre-training on a massive amount of unlabeled IHU data (Section III-D). The objective is to enable the model to learn the universal context and structural representations of IHU traffic data. This thoroughly pre-trained large model is defined as IHU-BERT.

Considering that the IHU-BERT model has a large number of parameters and high computational costs, making it unsuitable for direct deployment in resource-constrained environments and causing increased inference latency for traffic classification, this paper introduces a Knowledge Distillation module (Section III-E). In this module, we use the pre-trained IHU-BERT as the teacher model to guide the learning of a student model of the same architecture but with fewer parameters and layers. Through distillation training using a defined loss function, the knowledge from the teacher model is efficiently transferred to the student model, resulting in a more lightweight model. This distilled, lightweight, and efficient student model is defined as IHUD-BERT.

Finally, the distilled lightweight model, IHUD-BERT, is fine-tuned on specific, labeled datasets for downstream tasks (Section III-E). With only a small amount of labeled data and computational resources, the model can quickly adapt to specific tasks and make high-precision predictions.

B. Network Traffic Feature Extraction and Data Augmentation

Existing network traffic classification methods almost universally involve data payloads. To ensure robust classification independent of encryption artifacts, this paper uses only the IP packet header for network traffic classification. The IP packet header typically consists of a fixed length of 20 bytes, including fields such as the version number and header length. However, some of these fields, such as the source and destination IP addresses, constitute private user information and are not strongly correlated with the intrinsic characteristics of the traffic type itself. Therefore, this paper selects the first 12 bytes of each packet's IP header as the representation for that packet [24].

Simultaneously, to reduce collection costs and enhance the feature richness of samples, this paper introduces a sliding window mechanism for data augmentation. As specifically illustrated in Figure 3, we first extract session flows that share the same five-tuple (source/destination IP, source/destination port, and protocol number) from a large volume of raw network traffic covering various categories; for the k session flow, we denote it as $\mathbf{S}_k$. If this session flow contains L_k packets, then $\mathbf{S}_k$ can be formally represented as a packet sequence, as shown in Equation (1) Here, $p_{k,i}$ represents the i packet in session flow $\mathbf{S}_k (1 \le i \le L_k)$ and we then extract the first 12 bytes of its IP header as its feature representation, denoted as $\mathbf{h}(p_{k,i})$. This feature is a 12-byte vector, as shown in Equation (2). Therefore, the k session flow $\mathbf{S}_k$ can be converted into a feature vector sequence H_k, as shown in Equation (3).

$$\mathbf{S}_k = (p_{k,1}, p_{k,2}, \ldots, p_{k,L_k}) \tag{1}$$

$$\mathbf{h}(p_{k,i}) \in \mathbb{R}^{12} \tag{2}$$

$$\mathbf{H}_k = (\mathbf{h}(p_{k,1}), \mathbf{h}(p_{k,2}), \ldots, \mathbf{h}(p_{k,L_k})) \tag{3}$$

After obtaining the feature vector sequence $\mathbf{H}_k$ for a session flow, we employ a sliding window of length N_ω to sample from this sequence. This window slides backward by N_s packets at each step, where N_s is the stride. The j window ($j \ge 0$)captures a subsequence $\mathbf{W}_{k,j}$ which contains the feature vectors from the $(j \cdot N_s + 1)$ packet to the $(j \cdot N_s + N_\omega)$ packet; the N_ω packets are stacked vertically in order to form a feature matrix $\mathbf{X}'_{k,j}$, as shown in Equation (4).

$$\mathbf{X}'_{k,j} = \begin{pmatrix} \mathbf{h}(p_{k,(j \cdot N_s)+1}) \\ \mathbf{h}(p_{k,(j \cdot N_s)+2}) \\ \vdots \\ \mathbf{h}(p_{k,(j \cdot N_s)+N_w}) \end{pmatrix} \tag{4}$$

Here, $\mathbf{X}'_{k,j}$ is an $N_\omega \cdot 12$ dimensional matrix, where each row represents the 12-byte feature of a single packet, for a total of N_ω rows. This matrix is considered the intermediate representation of the IHU for the window. Before being input to the classification model, $\mathbf{X}'_{k,j}$ is flattened into a $1 \cdot (N_\omega \cdot 12)$ one-dimensional row vector, denoted as $\mathbf{X}_{k,j}$. Each window outputs one feature sample, the IHU; for a single session flow, the collection of all generated IHUs constitutes the set of feature samples for that flow, denoted as IHUs. These IHUs are then fed into the next module for dictionary construction.

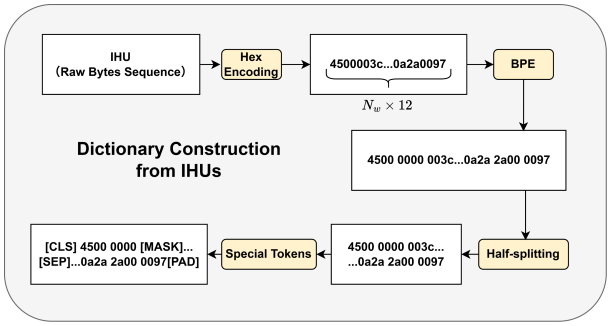


Fig. 4. IHU data flow diagram.

C. Dictionary Construction

The core task of the Dictionary Construction module is to convert the IHUs generated in the previous stage from raw byte sequence data into token sequences that the pre-trained model can understand and process. This process draws on ideas from NLP, "textualizing" the network traffic data to lay the foundation for subsequent pre-training and fine-tuning. As shown in Figure 4, the entire conversion process comprises four core components: Hex Encoding, Byte-Pair Encoding (BPE), Half-splitting, and Special Tokens.

1) Hex Encoding: The input IHU is a one-dimensional row vector formed by concatenating the first 12 bytes of multiple IP packet headers; it is fundamentally binary data. To facilitate subsequent tokenization while preserving the original byte information, the data first passes through the Hex Encoding module. This module converts each byte into two hexadecimal characters (e.g., the byte 01000101 is converted to the hexadecimal string '45'); after this step, the IHU is transformed into a hexadecimal string.

2) Byte-Pair Encoding: After obtaining the hexadecimal sequence, the BPE module is responsible for constructing the vocabulary and performing tokenization. Similar to how two adjacent bytes are combined into a basic unit in BERT, our method first merges adjacent characters in the hexadecimal sequence. Subsequently, the BPE algorithm iteratively counts the most frequently occurring adjacent byte pairs and merges them into a new, larger token.

Specifically, every two adjacent bytes (four hexadecimal characters) are combined into a base token. As shown in Figure 4, the hexadecimal sequence '4500003c' would be initially processed into units such as ['4500', '0000', '003c']. The BPE algorithm then learns further merging rules on this basis, ultimately generating a rich vocabulary. In this way, BPE strikes a balance between the non-semantic nature of raw bytes and the statistical properties of high-frequency byte combinations. Each generated token (IHUtoken) corresponds to a unique integer ID ranging from 0 to 65535; therefore, the maximum size of the base vocabulary, $|V|$, constructed by our model is 65,536.

3) Half-Splitting: To support the Same-origin IHU Prediction (SIP) task in the subsequent pre-training stage, the Half-splitting module evenly divides a single long IHU token sequence into two sub-sequences of equal length; these two sub-sequences are referred to as a sub-IHUtokens pair. The purpose of this design is to enable the model to learn to determine whether two consecutive traffic segments originate from the same sliding window sample, thereby capturing the continuity features of the traffic in its time series.

4) Special Tokens & Embedding: Finally, to meet the input requirements of the BERT model and to assist with subsequent training tasks, the Special Tokens module adds four types of special control tokens to the processed token sequence:

[CLS] (Classification): Added to the beginning of each sequence. During the fine-tuning stage, the final hidden state vector corresponding to this token in the Transformer's output layer serves as the aggregate representation of the entire sequence for downstream classification tasks.

[SEP] (Separator): Used to separate different sequence segments. This token is primarily used to mark sub-IHUtokens pairs, explicitly indicating the boundary between the two segments to the model.

[PAD] (Padding): Used to pad sequences. Since the sequences input to the model must have a uniform length, this token is appended to the end of a sequence until it reaches the predefined maximum length.

[MASK] (Mask): Used for masking. In the pre-training task of the Masked IHU Model (MIM), this token randomly replaces some of the tokens in the input sequence, and the model is required to predict the original masked tokens based on the context.

After being processed through all the steps mentioned above, the original IHU data stream is successfully converted into a formatted token sequence. These sequences are then passed through an Embedding layer, which includes Position Embedding and Segment Embedding. Position Embedding is used to capture the positional information of tokens within the sequence, preserving the inherent order and structure of the IP header fields. Segment Embedding assigns different embedding vectors to the two sub-sequences, helping the model to distinguish and understand their logical relationship. Finally, the token embedding, position embedding, and segment embedding are summed together to form the final representation that is input to the pre-trained model.

D. Pre-Training

To enable the model to learn deep, task-agnostic, and universal feature representations from network traffic data, we have designed a self-supervised pre-training stage. Traditional supervised learning methods are highly dependent on large-scale labeled datasets, which are often difficult to obtain in the field of network traffic analysis. Drawing inspiration from the success of BERT in NLP, we adopt the Pre-training and Fine-tuning approach. This approach first involves pre-training the model on a massive amount of unlabeled traffic data to grasp universal traffic patterns and structural knowledge; it is then fine-tuned on a small amount of labeled data for a specific downstream task to achieve superior performance.

To achieve this goal, we have designed two self-supervised pre-training tasks: the Masked IHU Model (MIM) and Same-origin IHU Prediction (SIP). These two tasks jointly encourage the model to learn the contextual and structural relationships of IHUs at different granularities.

1) Masked IHU Model (MIM): Standard language models are typically unidirectional (either left-to-right or right-to-left), a structure that limits the contextual information the model can leverage during pre-training. To construct a deep bidirectional

model capable of simultaneously utilizing both left and right contextual information, this paper introduces the MIM model. This task is analogous to the Masked Language Model (MLM) in BERT.

In the MIM task, we randomly select 15% of the tokens from each IHU sequence for a masking operation. Doing so corrupts the original input and drives the model to predict the original value of the masked tokens based on their bidirectional context. However, this introduces a mismatch between the pre-training and fine-tuning stages, as the [MASK] token does not appear during fine-tuning. To mitigate this issue, we do not always replace the selected tokens with the [MASK] token. The specific replacement strategy is as follows: of the masked tokens, 80% are replaced with [MASK], 10% are replaced with a random token, and 10% remain unchanged. Through this strategy, the model can learn a distributed contextual representation for every token in the sequence, rather than merely relying on the presence of the [MASK] label. The model's objective is to minimize the cross-entropy loss to accurately predict the original value of the masked tokens. Let the input IHU sequence after masking be $\mathbf{X}$, a masked position be $f \in F$, and its true token be t_f. With model parameters θ, the loss function is as shown in Equation (5).

$$L_{MIM} = -\sum_{f \in F} \log P(t_f \mid \mathbf{X}; \theta) \tag{5}$$

2) Same-Origin IHU Prediction (SIP): Many important network traffic analysis tasks, such as session identification or anomaly detection, require an understanding not only of the internal structure of a single IHU sequence but also of the logical relationships and interaction patterns between different groups of packets. Inspired by the Next Sentence Prediction (NSP) task in BERT, we designed the SIP task to enhance the model's ability to capture these inter-packet-group dependencies and the underlying communication logic within a network flow.

The SIP task is formulated as a binary classification problem. First, we define how the input sequences are constructed. Since our sliding window mechanism already groups multiple packets into a single IHU, we treat two IHU sub-sequences as IHU_A and IHU_B. To construct the input for the model, these two sub-sequences are concatenated into a single sequence with special tokens in the format of "[CLS] IHU_A [SEP] IHU_B [SEP]".

During the training data generation, we create positive and negative samples at a 1:1 ratio. In 50% of the cases (positive samples), IHU_B is the actual next continuous IHU sub-sequence that immediately follows IHU_A in the sliding window sampling of the same original session flow. In the remaining 50% of cases (negative samples), IHU_B is an IHU sub-sequence randomly sampled from a completely different session flow. The physical objective of SIP is to force the model to predict whether two given IHU sub-sequences originate from the same session flow context.

Let the g-th pair of sub-IHU sequences be $\mathbf{I}_g^{(a)}$ and $\mathbf{I}_g^{(b)}$, combined as $\mathbf{I}_g = (\mathbf{I}_g^{(a)}, \mathbf{I}_g^{(b)})$ with the corresponding ground-truth label being $y_g \in \{0, 1\}$. Here, $y_g = 1$ indicates that the pair originates from the same session, while $y_g = 0$ indicates a different origin. The model uses the final hidden state corresponding to the [CLS] token, passing it through a linear classifier to output the probability. The SIP loss function L_{SIP} is calculated using binary cross-entropy, as shown in Equation (6). Through this task, the model learns high-level abstract knowledge regarding traffic continuity, protocol handshakes, and session structures.

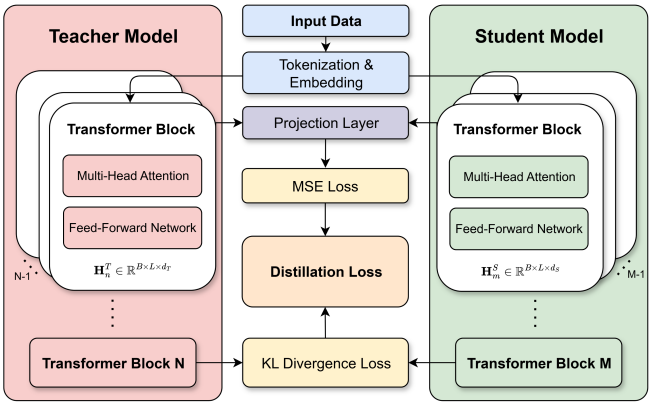


Fig. 5. The distillation process.

Finally, the overall pre-training objective of this paper is the sum of the MIM loss and the SIP loss, which are optimized simultaneously through multi-task learning. The total loss L_{task} is defined as shown in Equation (7). During the training phase, the gradients from both the token-level MIM task (focusing on local header syntax) and the sequence-level SIP task (focusing on global session logic) are backpropagated in parallel. This synergistic optimization allows the Transformer to comprehensively model both the micro-level packet features and the macro-level flow dynamics.

$$L_{SIP} = -\sum_{g=1}^{N} \log P(y_g \mid \mathbf{I}_g; \theta) \tag{6}$$

$$L_{task} = L_{MIM} + L_{SIP} \tag{7}$$

E. Distillation and Fine-Tuning

To efficiently transfer the rich knowledge embedded in the large teacher model (IHU-BERT) to the lightweight student model (IHUD-BERT), while also addressing the challenge that simpler models face in learning complex feature representations due to their limited parameters and layers, this paper proposes a hierarchical knowledge distillation strategy. This strategy designs different knowledge transfer objectives tailored to the characteristics of the output features at various levels of the Transformer; it not only focuses on aligning the prediction distribution of the final output layer but also delves into the model's intermediate hidden layers to ensure that the student model can learn the fine-grained internal representations formed by the teacher model during the feature extraction process. The specific method involves using a Mean Squared Error (MSE) loss on the intermediate layers to preserve precise feature alignment, and a Kullback-Leibler (KL) divergence loss on the final layer to capture output distribution information. This hierarchical distillation strategy can simultaneously ensure the precision of low-level features and the distributional consistency of high-level representations.

The specific distillation process is shown in Figure 5; the input IHU sequence data, after undergoing the same Tokenization and Embedding process, is fed into the teacher and student

models, respectively. Let the total number of layers in the teacher model be N, and the total number of layers in the student model be M.

Since the teacher and student models have different numbers of layers, this paper adopts a uniformly distributed layer mapping method to project the student model's hidden layers into the dimensional space of the teacher model, thereby resolving the dimension mismatch. For each intermediate layer $m \in \{1, \ldots, M-1\}$ of the student model $\mathbf{H}_m^S \in \mathbb{R}^{B\times L\times d_S}$, we compute the MSE between its mapped hidden state and the hidden state $\mathbf{H}_n^T \in \mathbb{R}^{B\times L\times d_T}$ of the corresponding teacher model layer $n \in \{1, \ldots, N-1\}$. Simultaneously, because the hidden dimension of the teacher (d_T) differs from that of the student (d_S), we introduce an intermediate-layer linear projection matrix $\mathbf{W}_m \in \mathbb{R}^{d_S\times d_T}$ to project the hidden states of the student into the same dimensional space as the teacher. To ensure that the loss is calculated only at valid, non-padded token positions, we introduce an attention mask matrix $\mathbf{M} \in \mathbb{R}^{B\times L}$. The definition for the intermediate layers loss L_{MSE} is as shown in Equation (8), where $\odot$ represents element-wise broadcast multiplication.

$$L_{MSE} = \frac{1}{M-1}\sum_{m=1}^{M-1} \| \mathbf{W}_m\mathbf{H}_m^S \odot \mathbf{M} - \mathbf{H}_n^T \odot \mathbf{M} \|_F^2 \tag{8}$$

Through the MSE, we compel the student model to precisely mimic the teacher model's internal representations in the feature space.

For the model's final output layer (i.e., the M layer), we utilize KL divergence to align the high-level feature representations. Specifically, the KL divergence is applied directly to the final-layer hidden state vectors(projected to the teacher's dimensional space) rather than the downstream task logits or token-level prediction heads. This method treats the high-dimensional feature vectors as activation distributions, encouraging the student to mimic the relative activation patterns and structural knowledge encoded within the teacher's feature space. KL divergence is an asymmetric metric that measures the difference between two probability distributions, and its definition is shown in Equation (9).This formula reflects the information loss incurred when approximating the teacher model's distribution P with the student model's distribution Q. A smaller KL divergence indicates that the student model is closer to the teacher model.

$$D_{KL}(P\|Q) = \sum_n P(n)\log\frac{P(n)}{Q(n)} \tag{9}$$

For the final layer output of the model proposed in this paper, we apply a softmax function with a temperature parameter τ and calculate the KL divergence between the soft label distributions, as detailed in Equations (10) and (11). Here, $\mathbf{W}_{final} \in \mathbb{R}^{d_S\times d_T}$ is the final layer output projection matrix. In knowledge distillation, the output of the teacher model, after being processed by a softmax function with a temperature coefficient τ, generates a smoother probability distribution than the original "hard labels" (one-hot vectors); this is known as "soft labels". When $\tau > 1$, the probability distribution becomes flatter, which allows the negative labels to also carry a certain amount of information. This strategy can amplify the relative differences between low-confidence classes, revealing the inter-class similarity information learned by the teacher model, which is the so-called "Dark Knowledge" [21]. The KL divergence loss can capture fine-grained information at the distribution level, allowing the teacher model to transfer more structured knowledge to the student model and ensuring that the student model's output probability distribution is similar to that of the teacher model.

$$\mathbf{P}_\tau^T = \text{softmax}\left(\frac{\mathbf{H}_N^T}{\tau}\right), \mathbf{P}_\tau^S = \text{softmax}\left(\frac{\mathbf{W}_{final}\mathbf{H}_M^S}{\tau}\right) \tag{10}$$

$$L_{KL} = \tau^2 D_{KL}(\mathbf{P}_\tau^T \| \mathbf{P}_\tau^S) \tag{11}$$

The intermediate layer MSE loss and the final layer KL loss are combined through a weighted sum to form the overall distillation loss, as shown in Equation (12). In the experiments, $\omega_1 + \omega_2 = 1$.

$$L_{distll} = \omega_1 \cdot L_{MSE} + \omega_2 \cdot L_{KL} \tag{12}$$

It is crucial to note that during the distillation phase, the student model is trained exclusively using the distillation loss $L_{distill}$, without incorporating the standard pre-training task loss. This design choice is grounded in two reasons. First, forcing a lightweight student model to simultaneously optimize for hard labels (MIM/SIP ground truth) and soft labels (teacher feature distributions) often leads to gradient conflicts, hindering stable convergence [25]. Second, given the significant capacity gap between the teacher (BERT-base) and the student (BERT-tiny), relying entirely on the robust contextual knowledge provided by the teacher allows the student to optimally utilize its limited parameters [26].

In the fine-tuning stage, we use the distilled, lightweight student model as a new pre-trained model. It is crucial to clarify the operational distinction between the training and inference phases regarding computational overhead. The knowledge transfer from Teacher to Student occurs exclusively during the offline training phase. Upon completion, the heavy Teacher model is immediately discarded, leaving only the lightweight Student model for the online inference pipeline. This ensures zero latency overhead during deployment, allowing the standalone student model to be efficiently fine-tuned for downstream tasks.

For a specific downstream classification task, we fine-tune the student model using the labeled data from that task. Specifically, the task data (which also undergoes IHU construction and tokenization) is input to the student model, and a simple classification layer is added on top of the model; the special [CLS] token is used for the multi-class classification task. The model's optimization objective is to minimize the cross-entropy loss for the standard classification task. Only a few epochs of training are required for the model to adapt to the specific task. Compared to training from scratch or directly fine-tuning a large model, fine-tuning the distilled student model can greatly reduce training costs and inference latency while ensuring high recognition accuracy, thus possessing greater practical application value.

IV. Experiments

In this chapter, we first introduce the selected pre-training and downstream datasets, and present the experimental environment and some of the model parameters (Section IV-A). Then, we conduct experiments with different models on

TABLE I
DOWNSTREAM DATASETS

Downstream Dataset	Classes	Total Packets
CIC200	200	2,405,944
CSTNET	120	581,709
CP-IOS	196	3,827,470
CP-Android	196	4,402,139

various datasets to demonstrate the effectiveness and generality of IHU-BERT and IHUD-BERT in different network traffic scenarios (Section IV-B). Afterward, we validate the effectiveness of the data augmentation method using sliding window sampling of network layer packet header bytes, and the designed loss function in the distillation process; through comparative experiments, we select the optimal sliding window size and the best parameters for distillation. By comparing with an undistilled model of the same size and parameters, we further illustrate the performance advantages of IHUD-BERT, and finally, we demonstrate the robustness of our framework under label-scarcity conditions through few-shot experiments (Section IV-C).

A. Experiment Setup

1) Pre-Training Dataset and Downstream Tasks: In this work, approximately 27.4 GB of unlabeled network traffic data is used for pre-training. This dataset consists of two parts: (1) The Android malware dataset [27] (CICAndMal2017), which is a network dataset for detecting malicious traffic, containing both malware and benign software traffic. This data was collected by running malware and benign applications on real smartphones, with a total size of approximately 17.3 GB. (2) The TLS 1.3 encrypted traffic collected from the China Science and Technology Network (CSTNET), known as CSTNET-TLS1.3 [5], with a total size of approximately 10.1 GB. The pre-training dataset includes a wide variety of network protocols; in addition to TCP/UDP, it also contains common network protocols such as Transport Layer Security, File Transfer Protocol, Hyper Text Transfer Protocol, as well as other encrypted protocols.

For the downstream tasks, to evaluate the effectiveness and generalization ability of IHU-BERT and IHUD-BERT, we selected four open-source, large-scale network traffic datasets. Their detailed statistics are shown in Table I: (1) The CICAndMal2017 dataset contains a total of 1700 application classes; we sorted them by the number of session flows and selected the top 200 applications to form the CIC200 dataset for our downstream task. (2) The CSTNET-TLS1.3 dataset contains 120 application classes; we constructed the downstream dataset, abbreviated as CSTNET, by randomly selecting a maximum of 5,000 packets from each flow. (3) We selected the Cross-Platform(IOS) and Cross-Platform(Android) open-source datasets [28]; these datasets were created from the top 100 applications from software stores in the US, China, and India, with each application running for three to ten minutes while receiving real user input. For each of the two datasets, we selected 196 application classes, naming them CP-IOS and CP-Android.

To conduct a systematic evaluation of the model, we categorized these datasets into two types based on their experimental roles. First, CIC200 and CSTNET share the same source domains as the pre-training corpus, assessing the model's adaptability within familiar environments. Second, CP-IOS and CP-Android serve as heterogeneous datasets external to the pre-training data, rigorously validating generalization across unseen distributions.

2) Data Pre-Processing: In this paper, we exclusively retain packets where the network layer is IPv4, filtering out traffic unrelated to the transmitted content, such as Address Resolution Protocol (ARP) packets. Crucially, to prevent data leakage and ensure rigorous evaluation, we employ a strict "inter-flow splitting" strategy during the fine-tuning stage. Unlike random packet-level splitting, we partition each flow into one of the training, validation, and testing sets at a ratio of 8:1:1 based on complete session flows. This ensures that all packets belonging to a specific session (identified by the five-tuple) reside exclusively within a single subset, thereby forcing the model to learn generalized behavioral features rather than memorizing session-specific artifacts. Subsequently, the sliding window mechanism is applied to the packet sequences within these already-partitioned flows to generate the final input feature samples (IHUs).

3) Evaluation Metrics and Implementation Details: To comprehensively evaluate and compare model performance, we employ four standard metrics: Accuracy (AC), Precision (PR), Recall (RC), and F1-score (F1). To ensure a fair evaluation across all classes, especially considering the inherent long-tail distribution (class imbalance) typical in real-world application traffic, the PR, RC, and F1 are calculated using the macro-average method. This treats all classes equally, providing a more rigorous assessment of the model's generalization capabilities. To mitigate bias from a single data partition and evaluate model stability, experiments are executed across three independent runs with varying random seeds. Performance is reported as the Mean $\pm$ Standard Deviation, while sensitivity analysis maintain a single fixed seed. Additionally, to measure the model's inference speed, we introduce Inference Time (IT) as a speed evaluation metric; this metric represents the time taken for each group of IHU from input to output.

The selected teacher model is BERT-base, and the student model is BERT-tiny. When $|V| = 65536$, the maximum input sequence length for both models is 512. Detailed model parameters for the teacher and student models are shown in Table II. For the comparative experiments presented in Section IV-B, we utilized the optimal hyperparameter configuration identified in our subsequent sensitivity analysis (Section IV-C). Specifically, unless otherwise stated, the sliding window size is set to $N_\omega = 12$, the distillation temperature is $\tau = 2$, and the intermediate layer loss weight is $\omega_1 = 0.7$ (with $\omega_2 = 0.3$).

In the pre-training stage, the batch size is set to 32, the total number of training steps is 500,000, and the learning rate is set to 2×10^{-5}. In the fine-tuning stage, we use the AdamW optimizer for training with a batch size of 32, 10 epochs, a learning rate of 2×10^{-5}, and a Dropout ratio of 0.5. All experiments were implemented based on Python 3.8 (Ubuntu 20.04), CUDA 11.3, PyTorch 1.10.0, and the UER toolkit [29]. The training and GPU inference

TABLE II
MODEL PARAMETER INFORMATION

Model	emb_size	feedforward_size	hidden_size	heads_num	layers_num	#params	#flops
BERT-base	768	3072	768	12	12	136.4M	91.8G
BERT-tiny	128	512	128	2	2	8.9M	0.54G

experiments were conducted on an NVIDIA RTX 4090D (24GB) GPU, while the CPU inference benchmark was performed on an Intel Xeon Platinum 8481C processor to evaluate deployability. For inference timing, we employed a rigorous measurement protocol: a warm-up phase of 10 batches was followed by 10 independent repeated runs to calculate the Mean $\pm$ Standard Deviation. We evaluated two scenarios: real-time latency with a batch size of 1 and high-throughput performance with a batch size of 32/64/128. The recorded time strictly captures the model's forward pass latency, excluding data loading and I/O overhead.

B. Experiment Results

To rigorously assess the model's performance across multiple evaluation criteria, we conduct comparative experiments on the four selected large-scale downstream network traffic datasets. We compare our proposed IHU-BERT and its distilled student model, IHUD-BERT, with two baseline models: ET-BERT and LEXNet. Additionally, to validate the effectiveness of our proposed data processing method, we also introduce the IHU-LEXNet model.

These two models are chosen as baselines because they represent two important directions in current large-scale traffic classification research. ET-BERT is one of the pioneering works that successfully applied the pre-training paradigm to this field, while LEXNet is currently the only representative model capable of classifying up to 200 classes.

ET-BERT: This model learns contextual datagram-level representations from encrypted traffic payloads [5]. To ensure a strictly fair comparison, we did not rely on the pre-trained weights provided by the original authors. Instead, we re-implemented the pre-training process from scratch. Specifically, we extracted payload sequences formatted as BURSTs (defined as continuous sequences of time-adjacent packets traveling in the same direction) from the same raw traffic datasets (CICAndMal2017 and CSTNET-TLS1.3) used for our IHU-BERT. Strictly adhering to the original methodology, we constructed the pre-training corpus by aggregating time-adjacent packets into BURSTs. The vocabulary was generated using BPE on bi-gram hexadecimal units, resulting in a vocabulary size of 65,536. Input sequences were formatted with special tokens and truncated or padded to a fixed length of 512 tokens. Furthermore, we strictly applied the inter-flow splitting strategy during the fine-tuning stage, ensuring that all packets from the same session reside exclusively in one data partition to prevent data leakage and session artifact memorization. Both models utilize the BERT-Base architecture and share identical configurations across the pre-processing, pre-training, and fine-tuning phases.

LEXNet: To provide a robust comparison against non-payload methods, we selected LEXNet [16] as our strong metadata-based baseline. LEXNet is a state-of-the-art lightweight CNN explicitly designed for encrypted traffic classification without relying on payloads. Instead, it natively utilizes session metadata. In our experiments, we strictly adhered to the original protocol, modeling the traffic as a Multivariate Time Series (MTS) by extracting two key statistical metadata features—packet size (bytes) and packet direction—from the first 20 packets of each flow. This results in a standardized input dimension of 20×2 for every sample.

IHU-LEXNet: To isolate and validate the effectiveness of our proposed IHU feature representation, and to provide a direct header-native baseline, we introduced IHU-LEXNet. This variant replaces the input layer of the standard LEXNet to accept IHUs while keeping the remaining backbone architecture unchanged. Specifically, distinct from the standard LEXNet which processes a 20×2 MTS matrix of statistical features (packet sizes and directions), IHU-LEXNet is adapted to ingest a 12×12 input matrix. This input corresponds to the optimal sliding window of 12 packets, where each packet is represented by its first 12 IP header bytes. Crucially, this raw byte input is modeled as a single-channel image, allowing us to leverage the exact same 2D CNN backbone as the baseline. To ensure a strictly fair comparison, both LEXNet and IHU-LEXNet were trained under identical hyperparameter settings derived from the original LEXNet configuration. Both models utilized a batch size of 64 and followed the same differentiated learning rate schedule (e.g., 10^{-3} for feature extraction layers and 10^{-1} for prototype vectors). Additionally, the coefficients for the combined loss function (incorporating cross-entropy, clustering, and separation losses) were kept strictly consistent. Moreover, both models followed the same strict inter-flow splitting protocol to ensure that any performance gap is attributable solely to the difference in feature representation.

Comparing IHU-BERT with LEXNet and IHU-LEXNet allow us to rigorously evaluate whether our proposed raw IP header pre-training yields superior representations compared to explicit statistical metadata engineering.

The detailed results of all experiments are shown in Tables III and IV.

A comparison of the performance between LEXNet and IHU-LEXNet clearly shows the superiority of our proposed data processing method. On all datasets, by only replacing the input data from its original format with the IHUs generated by our method, IHU-LEXNet achieves a substantial improvement in performance metrics over LEXNet. For example, on the CIC200 dataset, the AC significantly increases from 0.5908 to 0.7828. This provides strong evidence that using IP headers and a sliding window mechanism can effectively extract key classification features from traffic. Furthermore, our proposed IHU-BERT model exhibits state-of-the-art performance across all four datasets on all evaluation metrics (AC, PR, RC, F1).

As the distilled student model, IHUD-BERT achieves an optimal balance between classification accuracy and inference efficiency. As shown in Table III, its accuracy shows only a slight decrease compared to the teacher model (e.g., a 3.2% drop on CIC200). More importantly, the efficiency analysis in

TABLE III
COMPARATIVE EXPERIMENTAL RESULTS ON DIFFERENT DATASETS

(a) Source Domain Datasets (CIC200 & CSTNET)

Method	CIC200				CSTNET			
	AC(%)	PR(%)	RC(%)	F1(%)	AC(%)	PR(%)	RC(%)	F1(%)
LEXNet (Statistical)	59.08 ± 0.32	47.30 ± 0.38	47.88 ± 0.32	47.59 ± 0.35	42.10 ± 0.25	39.78 ± 0.28	39.11 ± 0.26	39.44 ± 0.26
IHU-LEXNet	78.28 ± 0.25	71.24 ± 0.26	70.90 ± 0.26	71.07 ± 0.26	75.10 ± 0.22	71.22 ± 0.20	71.86 ± 0.20	71.54 ± 0.20
ET-BERT (Payload)	13.76 ± 0.51	13.64 ± 0.55	13.62 ± 0.58	13.63 ± 0.56	11.45 ± 0.42	11.25 ± 0.46	11.32 ± 0.46	11.28 ± 0.46
IHU-BERT	**94.84 ± 0.01**	**92.66 ± 0.01**	**92.08 ± 0.01**	**92.37 ± 0.01**	**92.86 ± 0.01**	**90.14 ± 0.01**	**89.86 ± 0.01**	**90.00 ± 0.01**
IHUD-BERT	91.64 ± 0.11	90.12 ± 0.11	90.08 ± 0.11	90.10 ± 0.11	89.65± 0.12	86.85 ± 0.15	86.50 ± 0.12	86.72 ± 0.13

(b) Heterogeneous Datasets (CP-IOS & CP-Android)

Method	CP-IOS				CP-Android			
	AC(%)	PR(%)	RC(%)	F1(%)	AC(%)	PR(%)	RC(%)	F1(%)
LEXNet (Statistical)	61.86 ± 0.35	60.52 ± 0.40	59.48 ± 0.32	59.99 ± 0.36	61.32 ± 0.38	59.46 ± 0.42	59.46 ± 0.35	59.46 ± 0.38
IHU-LEXNet	80.08 ± 0.28	76.82 ± 0.25	75.66 ± 0.26	76.24 ± 0.25	80.12 ± 0.26	77.02 ± 0.28	76.18 ± 0.24	76.60 ± 0.26
ET-BERT (Payload)	13.52 ± 0.52	12.45 ± 0.58	12.85 ± 0.55	12.65 ± 0.56	13.38 ± 0.48	12.82 ± 0.52	12.52 ± 0.60	12.67 ± 0.55
IHU-BERT	**96.40 ± 0.01**	**93.60 ± 0.01**	**92.88 ± 0.01**	**93.24 ± 0.01**	**96.76 ± 0.01**	**93.76 ± 0.02**	**93.04 ± 0.01**	**93.40 ± 0.01**
IHUD-BERT	92.22 ± 0.14	90.12 ± 0.12	90.24 ± 0.15	90.18 ± 0.14	92.60 ± 0.15	90.15 ± 0.13	90.28 ± 0.12	90.21 ± 0.13

TABLE IV
END-TO-END INFERENCE COST AND MEMORY FOOTPRINT ON DIFFERENT DEVICES

(a) Hardware: GPU (NVIDIA RTX 4090D)

Model	**Peak Memory (VRAM)**	**End-to-End Latency (μs / sample)**			
		Batch=1	Batch=32	Batch=64	Batch=128
IHU-LEXNet	78.23 MB	2075.55 ± 2.24	1587.48 ± 0.12	1580.82 ± 0.04	1579.05 ± 0.05
IHU-BERT	1340.14 MB	6322.49 ± 76.89	2282.27 ± 0.73	1653.82 ± 1.21	2419.35 ± 1.80
IHUD-BERT	**123.85 MB**	**2884.63 ± 5.38**	**1607.55 ± 0.75**	**1589.42 ± 0.18**	**1586.52 ± 0.10**

(b) Hardware: CPU (Intel Xeon Platinum 8481C)

Model	**Peak Memory (RAM)**	**End-to-End Latency (μs / sample)**			
		Batch=1	Batch=32	Batch=64	Batch=128
IHU-LEXNet	38.45 MB	3684.54 ± 33.63	4409.70 ± 38.20	5084.54 ± 45.27	7671.58 ± 150.60
IHU-BERT	869.24 MB	35497.19 ± 288.59	35742.94 ± 778.79	34633.33 ± 760.98	54324.16 ± 622.17
IHUD-BERT	**63.75 MB**	**6330.31 ± 65.61**	**8275.31 ± 102.94**	**9293.14 ± 360.80**	**10381.94 ± 232.74**

Table IV demonstrates the practical deployability of IHUD-BERT. To comprehensively evaluate real-world performance, we report the end-to-end inference latency, which strictly encompasses the entire preprocessing pipeline (including IHU creation, hex encoding, and BPE tokenization) alongside the model's forward pass. On the GPU, the end-to-end inference delay per sample achieves remarkable real-time latency (batch size= 1) of only 2884.63 μs, while it drops to 1589.42 μs under high-load settings (batch size= 64). There seems a seemingly strange phenomenon: when the batch size is 1, the average GPU run time per sample is longer when the batch size is larger. The reasons may be as follows. To enable the GPU to execute tasks like the model's forward pass, it must first go through a sequence of steps known as launch overhead or Kernel Launch Latency. Consequently, for a very small task like batch size= 1, this overhead cannot be amortized across multiple samples. In contrast, when the batch size is 32, 64, or 128, the non-computation overhead averaged per sample becomes relatively small. Hence, the average GPU inference delay seems shorter when batch size is larger than 1.

Furthermore, Table IV reports the peak memory usage during inference, providing key insights into hardware requirements. On a standard CPU, IHUD-BERT requires only 63.75 MB of RAM, which closely matches the theoretical footprint of its 8.9M parameters and enables deployment on resource-constrained edge devices. On the GPU, the peak memory usage reaches 123.85 MB due to hardware-specific factors such as workspace pre-allocation, CUDA context initialization, and memory alignment for parallel computation. Nevertheless, this remains significantly lower than the 1340.14 MB required by the teacher model (IHU-BERT), demonstrating the lightweight nature of IHUD-BERT.

Crucially for edge deployment, on a standard CPU, IHUD-BERT reduces the single-sample end-to-end latency to 6330.31 μs ($\approx$ 6.33 ms), significantly outperforming the teacher model. Regarding scalability, we observed a non-linear latency degradation for the heavy teacher model (IHU-BERT) at large batch size (batch size= 128). This anomaly occurs because the large batch size exceeds the limited capacity of the hardware's high-speed memory. Specifically, the GPU is

forced to rely on slower external data transfers, while the CPU faces bottlenecks in retrieving data fast enough from the main memory. In contrast, the lightweight IHUD-BERT avoids these memory walls. It is worth noting that for the student model on the CPU, the amortized time for batch size= 64 is slightly higher than the single-sample latency due to the overhead of parallel scheduling, further confirming that IHUD-BERT is efficient enough to be deployed in a serial, real-time manner.

In experiments on the CP-IOS and CP-Android datasets—two downstream datasets not included in the pre-training data—both IHU-BERT and IHUD-BERT also demonstrate excellent performance; this indicates that by pre-training on IHUs, the model can learn more robust and generalizable traffic feature representations.

Overall, although IHU-BERT achieves the highest scores on classification metrics, IHUD-BERT is undoubtedly the optimal choice for real-world environments. It significantly reduces both the end-to-end inference latency to the millisecond level and the memory footprint to tens of megabytes on generic CPUs, with almost no loss in accuracy, satisfying the stringent requirements for rapid responses in network environments. To further investigate the key factors affecting model performance and to conduct a more in-depth analysis of our proposed framework, in the next section, we perform detailed parameter selection and comparison experiments on the sliding window size and the key hyperparameters in the distillation process.

C. Sensitivity Analysis

This section presents a sensitivity analysis to further investigate the key factors affecting model performance. The first group investigates the impact of IHUs generated with different sliding window sizes on the final classification accuracy of the model, while also validating the effectiveness of the distillation process. The second group examines the influence of different hyperparameters during the distillation process on the student model's classification accuracy. The third group evaluates the robustness of the proposed framework under label-scarcity conditions through a series of few-shot experiments.

1) Sliding Window Parameter Selection: To quantitatively evaluate the impact of the sliding window size on model performance and to validate the effectiveness of our proposed data augmentation method, this section presents a set of systematic sensitivity analysis. As the core mechanism for extracting IHUs from session flows, the size of the sliding window (i.e., the number of packets contained within it, N_ω) directly determines the richness of the contextual information embedded in each training sample. Theoretically, a suitable window size should strike a balance between capturing sufficient dynamic sequence features and avoiding the introduction of redundant noise.

To investigate the optimal value for this parameter, we conduct experiments using a series of discrete window sizes, N_ω ={1,3,5,7,9,10,11,12,13,14,15,20}, while the sliding stride N_s is fixed to 1 to ensure the most fine-grained sampling of the session flow. The experiments are conducted on three models: IHU-BERT (base), IHU-BERT (tiny) (without distillation), and the distilled IHUD-BERT, for which the distillation hyperparameters adopt the optimal configuration determined in Section IV-C.2.

TABLE V
SENSITIVITY ANALYSIS ON SLIDING WINDOW SIZE

Sliding Window Size	IHU-BERT(base)	IHU-BERT(tiny)	IHUD-BERT
$N_\omega = 1$	0.7832	0.6982	0.7142
$N_\omega = 3$	0.8350	0.7246	0.7633
$N_\omega = 5$	0.8799	0.8121	0.8436
$N_\omega = 7$	0.8808	0.8173	0.8476
$N_\omega = 9$	0.8925	0.8309	0.8611
$N_\omega = 10$	0.8953	0.8309	0.8636
$N_\omega = 11$	0.9274	0.8711	0.8966
$N_\omega = 12$	**0.9484**	**0.8862**	**0.9174**
$N_\omega = 13$	0.9478	0.8853	0.9166
$N_\omega = 14$	0.9477	0.8846	0.9160
$N_\omega = 15$	0.9469	0.8801	0.9072
$N_\omega = 20$	0.9445	0.8781	0.9071

The experimental results are shown in Table V. It is observed that as the window size increases from 1 to 12, the classification accuracy of all models shows an increasing trend. Taking IHU-BERT (base) as an example, its accuracy significantly increases from 0.7832 at $N_\omega = 1$ to a peak of 0.9484 at $N_\omega = 12$. Similarly, the distilled student model, IHUD-BERT, achieves its optimal performance of 0.9174 at this window size. This phenomenon provides strong evidence that extending the context length effectively enhances the model's ability to discriminate traffic patterns, as longer sequences enable the model to capture more complex temporal dependencies between packets.

Notably, the performance peaks at $N_\omega = 12$, indicating that this length provides the model with nearly saturated effective feature information. However, when the window size continues to increase beyond 12 (e.g., $N_\omega = 13, 14, 15, 20$), the model's accuracy exhibits a saturation phenomenon and even a slight decline. For instance, the accuracy of IHU-BERT (base) drops slightly to 0.9445 at $N_\omega = 20$. This suggests that excessively long windows may introduce redundant information or noise irrelevant to the current classification task, which interferes with the model's feature extraction process. This phenomenon is common in sequence modeling [30], where overly long inputs can dilute the model's attention mechanisms. Based on this analysis, we determine that the optimal sliding window size is $N_\omega = 12$; this configuration achieves the best trade-off between ensuring information integrity and maintaining classification precision, and is therefore applied in all subsequent experiments.

Based on the analysis above, we determine that the optimal sliding window size is 12; this configuration achieves the best trade-off between ensuring information integrity and enhancing the model's classification accuracy, and is applied in all subsequent experiments.

2) Distillation Parameter Selection: To validate the effectiveness of our proposed distillation method and identify the optimal hyperparameter configuration, we conducted a comprehensive sensitivity analysis. We investigated the impact of two key hyperparameters on the student model's performance: the distillation temperature τ, and the intermediate layer loss weight ω_1 (where $\omega_2 = 1 - \omega_1$). In the experimental setup, the sliding window size was fixed at the optimal value of $N_\omega = 12$. We performed a grid search over the following parameter spaces: $\tau \in \{2, 3, 4\}$, and $\omega_1 \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$.

TABLE VI
SENSITIVITY ANALYSIS ON DISTILLATION PARAMETERS WITH WEIGHTS

ω_1	ω_2	$\tau = 2$	$\tau = 3$	$\tau = 4$
0.1	0.9	0.9168	0.9165	0.9162
0.3	0.7	0.9161	0.9147	0.9138
0.5	0.5	0.9161	0.9166	0.9156
0.7	0.3	**0.9174**	0.9172	0.9169
0.9	0.1	0.9131	0.9136	0.9121

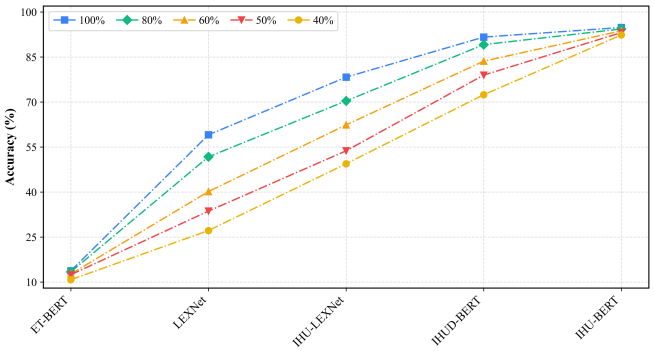


Fig. 6. Comparison results on few-shot CIC200.

As shown in Table VI, by performing a grid search on the distillation weights, we found that the model reaches a peak accuracy of 0.9174 when $\omega_1 = 0.7$ (with $\omega_2 = 0.3$). This suggests that placing a slightly higher emphasis on aligning the intermediate feature representations (L_{MSE}) enables the student model to better capture the complex structural patterns encoded by the teacher, effectively complementing the probability distribution knowledge (L_{KL}) from the final layer. Consequently, we select $\tau = 2$, and $\omega_1 = 0.7$, $\omega_2 = 0.3$ as the optimal distillation hyperparameter configuration for the IHUD-BERT model.

3) Impact of Labeled Data Scarcity: To validate the effectiveness and robustness of our proposed framework under label-scarcity conditions, we design comparative experiments using varying proportions of training data. Specifically, we randomly sample 80%, 60%, 50%, and 40% of the original labeled training instances from the CIC200 dataset while strictly maintaining the identical, unseen test set for evaluation. As illustrated in Figure 6, the comparative results clearly demonstrate that our pre-training and distillation paradigm is significantly more resilient to the reduction in data size.

The teacher model, IHU-BERT, exhibits remarkable robustness, with its accuracy merely degrading from 94.84% to 92.35% when the data scale shrinks to 40%. Notably, our distilled lightweight model, IHUD-BERT, maintains an accuracy of 72.44% under the extreme 40% few-shot setting, which still comprehensively outperforms the baseline LEXNet even when LEXNet utilizes 100% of the training data. In contrast, traditional supervised methods that lack pre-trained knowledge suffer from substantial performance deterioration. For example, the accuracy of LEXNet plunges from 59.08% to 27.17% as the sample size is reduced. This empirical evidence indicates that extracting invariant behavioral features from IP headers via unsupervised pre-training empowers the model to learn generalized representations, thereby mitigating the label-scarcity bottleneck in real-world traffic classification.

V. CONCLUSION

To address the challenges of data scarcity, inference efficiency, and generalization failures caused by encryption artifacts, this paper proposes an integrated traffic classification framework based on pre-training and knowledge distillation. The core innovation begins with feature representation. We identify that encrypted payloads in modern protocols introduce session-specific encryption artifacts that hinder generalization across sessions. In response, we design the IP Head Unit (IHU), which relies solely on IP packet headers. To specifically tackle the problem of labeled data scarcity, we employ a sliding window mechanism as a strategic data augmentation technique. By generating multiple overlapping samples from a single traffic flow, this method significantly enriches the training corpus, allowing the model to learn effectively even with limited raw data. Furthermore, this approach captures stable traffic behavioral patterns that are invariant to encryption keys. This ensures that the model generalizes well even under strict inter-flow splitting settings, where traditional payload-based methods fail. Building on this robust foundation, we pre-train a large teacher model, IHU-BERT, to learn universal representations and then employ hierarchical knowledge distillation to transfer this knowledge into a lightweight student model, IHUD-BERT. Experimental results confirm that our solution effectively overcomes the limitations of session artifacts in encrypted traffic, maintaining high classification accuracy where baselines collapse. Furthermore, the distilled student model significantly improves inference efficiency, providing a practical path for deploying high-performance, robust models in real-world network environments that require both precision and speed.

In summary, IHUD-BERT provides an efficient and robust solution for large-scale encrypted traffic classification. Although the model has achieved superior performance across multiple heterogeneous benchmark datasets, the inherent volatility of header fields in non-stationary network environments remains a critical area for future research. Future work will leverage broader and more diverse real-world datasets to further refine adaptive normalization techniques, aiming to more effectively decouple intrinsic application behaviors from transient, path-induced regularities. Additionally, we plan to extend the IHU mechanism to IPv6 architectures to ensure long-term reliability in next-generation network environments.

REFERENCES

[1] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.

[2] O. Salman, I. H. Elhajj, A. Kayssi, and A. Chehab, "A review on machine learning–based approaches for internet traffic classification," *Ann. Telecommun.*, vol. 75, nos. 11–12, pp. 673–710, Dec. 2020.

[3] X. Ren, H. Gu, and W. Wei, "Tree-RNN: Tree structural recurrent neural network for network traffic classification," *Expert Syst. Appl.*, vol. 167, Apr. 2021, Art. no. 114363.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, 2019, pp. 4171–4186.

[5] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.

[6] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.
[7] K. Boakye-Boateng, A. A. Ghorbani, and A. H. Lashkari, "Securing substations with trust, risk posture, and multi-agent systems: A comprehensive approach," in *Proc. 20th Annu. Int. Conf. Privacy, Secur. Trust (PST)*, Aug. 2023, pp. 1–12.
[8] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.
[9] A. S. Iliyasu and H. Deng, "Semi-supervised encrypted traffic classification with deep convolutional generative adversarial networks," *IEEE Access*, vol. 8, pp. 118–126, 2020.
[10] N. Williams, S. Zander, and G. Armitage, "A preliminary performance comparison of five machine learning algorithms for practical IP traffic flow classification," *SIGCOMM Comput. Commun. Rev.*, vol. 36, no. 5, pp. 5–16, Oct. 2006.
[11] S. Kumar, S. Dharmapurikar, Y. Fang, P. Crowley, and J. Turner, "Algorithms to accelerate multiple regular expressions matching for deep packet inspection," in *Proc. Conf. Appl.*, vol. 36. New York, NY, USA: Association for Computing Machinery, 2006, pp. 339–350.
[12] L. Xu, X. Zhou, Y. Ren, and Y. Qin, "A traffic classification method based on packet transport layer payload by ensemble learning," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2019, pp. 1–6.
[13] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, Mar. 2016, pp. 439–454.
[14] S. Rezaei, B. Kroencke, and X. Liu, "Large-scale mobile app identification using deep learning," *IEEE Access*, vol. 8, pp. 348–362, 2020.
[15] C. Wang, A. Finamore, L. Yang, K. Fauvel, and D. Rossi, "AppClassNet: A commercial-grade dataset for application identification research," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 52, no. 3, pp. 19–27, Jul. 2022.
[16] K. Fauvel, F. Chen, and D. Rossi, "A lightweight, efficient and explainable-by-design convolutional neural network for internet traffic classification," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2023, pp. 4013–4023.
[17] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
[18] L. Z. Albert, "A lite BERT for self-supervised learning of language representations," 2019, *arXiv:1909.11942*.
[19] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, "ELECTRA: Pre-training text encoders as discriminators rather than generators," 2020, *arXiv:2003.10555*.
[20] Md. S. Towhid and N. Shahriar, "Encrypted network traffic classification using self-supervised learning," in *Proc. IEEE 8th Int. Conf. Netw. Softwarization (NetSoft)*, Jun. 2022, pp. 366–374.
[21] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
[22] X. Jiao et al., "TinyBERT: Distilling BERT for natural language understanding," 2019, *arXiv:1909.10351*.
[23] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.
[24] Y. Hu, Z. Zeng, J. Song, L. Xu, and X. Zhou, "Online network traffic classification based on external attention and convolution by IP packet header," *Comput. Netw.*, vol. 252, Oct. 2024, Art. no. 110656.
[25] Z. Hayder, A. Cheraghian, L. Petersson, and M. Harandi, "MoKD: Multi-task optimization for knowledge distillation," 2025, *arXiv:2505.08170*.
[26] B. Peng, J. Lu, G. Zhang, and Z. Fang, "Rethinking knowledge distillation: A mixture-of-experts perspective," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 10969–10979.
[27] A. H. Lashkari, A. F. A. Kadir, L. Taheri, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark Android malware datasets and classification," in *Proc. Int. Carnahan Conf. Secur. Technol. (ICCST)*, Oct. 2018, pp. 1–7.
[28] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, Art. no. 24412, doi: 10.14722/ndss.2020.24412.
[29] Z. Zhao et al., "UER: An open-source toolkit for pre-training models," 2019, *arXiv:1909.05658*.
[30] H. I. Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Müller, "Deep learning for time series classification: A review," *Data Mining Knowl. Discovery*, vol. 33, no. 4, pp. 917–963, 2019.

Yahui Hu received the Ph.D. degree. She is currently an Associate Professor and a Master's Supervisor with China University of Mining and Technology-Beijing, Beijing, China. Her main research interests include data center network transmission optimization, edge computing and edge intelligence, computing power networks, network traffic analysis and optimization, and 6G.

Tiancun Deng is currently pursuing the master's degree with China University of Mining and Technology-Beijing, Beijing, China. His main research interests include encrypted network traffic classification.

Junping Song received the Ph.D. degree. She is currently an Assistant Researcher with the Computer Network Information Center, Chinese Academy of Sciences, Beijing, China. Her main research interests include computing power networks and network artificial intelligence.

Pengfei Fan is currently a Senior Engineer and a Master's Supervisor with the Computer Network Information Center, Chinese Academy of Sciences, Beijing, China. His main research interests include future network architectures and technologies, and computing power and network convergence technologies and applications.

Xu Zhou received the Ph.D. degree. He is currently a Professor and a Ph.D. Supervisor with the Computer Network Information Center, Chinese Academy of Sciences, Beijing, China. His main research interests include computer network architectures, 5G mobile communications, and network artificial intelligence technologies.